

Cours 2 ECAABD MAABOUT

La Classification

I- Classification VS Prédiction

Classification

- **Définition** : La classification consiste à **attribuer une donnée à une classe prédéfinie**.
- On a un **ensemble d'apprentissage (training set)** où les classes sont déjà connues (par ex. spam ou non spam).
- L'algorithme apprend à reconnaître les caractéristiques qui différencient les classes.
- **Sortie attendue** : une **étiquette** (catégorie qualitative).

Exemple : prédire si un client va **acheter** (oui/non), si un mail est **spam** ou **non spam**, si une image est un **chat** ou un **chien**.

Prédiction

- **Définition** : La prédiction vise à **estimer une valeur numérique continue** à partir des données.
- On utilise également un ensemble d'apprentissage, mais cette fois avec des valeurs numériques connues pour entraîner le modèle.
- **Sortie attendue** : une **valeur numérique** (quantitative).

Exemple : prédire le **chiffre d'affaires** du mois prochain, le **prix d'un logement**, la **température** demain.

Différence clé

- **Classification** = prédire une **catégorie** (discrète, qualitative).
- **Prédiction** = prédire une **valeur numérique** (continue, quantitative).

II- Construction du modèle : 2 étapes

a. Construction du modèle

Chaque **tuple** (exemple) est supposé appartenir à une **classe** comme spécifié par le label de l'attribut "**Classe**".

→ **Label** : Valeur de sortie attendue dans un problème de classification.

Les **données** sont partagées en deux **sous-ensembles** :

- Le **premier sous-ensemble** sert à **construire le modèle**.
- Le **modèle** peut être représenté sous forme de **règles de classification**, d'**arbres de décision**, etc.

b. Estimation de la pertinence

On utilise le **deuxième sous-ensemble** pour **évaluer le modèle**.

- On compare les **labels de classe réels** des **tuples** avec ceux **prédits** par le **modèle**.
- Le **pourcentage** de **tuples correctement classifiés** par le **modèle** constitue une mesure de la **précision**.

a. Etape 1 : Construction du modèle

i. Définir la variable cible (Label)

- On choisit ce que le modèle doit **prédire**.
- Ici, c'est l'attribut « **Titulaire** » (valeurs possibles : **oui** ou **non**).
- C'est le **label**.

ii. Sélectionner les données d'apprentissage (Training Data)

- On constitue un tableau avec des **exemples passés** (les individus et leurs caractéristiques).
- Exemple simplifié :

Nom	Grade	Années	Titulaire
David	Assistant	3	non
Marie	Assistant	7	oui
Jean	Professeur	2	oui
Jim	Prof. Associé	6	oui
Pierre	Assistant	3	non
Anne	Prof Associé	3	non

- Ici, **Grade** et **Années** sont les **variables explicatives (features)**.
- **Titulaire** est la **variable à prédire (label)**.

iii. Choisir un algorithme de construction de modèle

- L'**algorithme** analyse les **données d'apprentissage** pour trouver une **règle générale**.
- Dans cet exemple, il en déduit une **règle logique simple** :

SI **Grade** = 'Professeur' **OU** **Années** > 6

ALORS **Titulaire** = 'oui'

SINON **Titulaire** = 'non'

Résumé de la première étape

La **première étape** consiste à :

- **Identifier le label** (« Titulaire »)
- **Utiliser les données d'apprentissage** avec les **variables explicatives (Grade, Années)**
- **Appliquer un algorithme** pour construire une **règle** qui explique au mieux la relation entre les **variables** et le **label**

b. Etape 2 : Vérification de la pertinence

i. Tester la pertinence du modèle

- Le **modèle** construit précédemment est **appliqué** au **jeu de test**.
- La **règle logique** utilisée est :

SI Grade = 'Professeur' **OU Années** > 6

ALORS Titulaire = 'oui'

SINON Titulaire = 'non'

- Le **modèle** prédit correctement **Tom, George, et Josephe**, mais se trompe sur **Elise**.

ii. Calculer la pertinence du modèle

- La **pertinence** (ou **accuracy**) correspond au pourcentage de **prédictions correctes**.
- Ici :

Pertinence = 75 %

Résumé de la deuxième étape

La **deuxième étape** consiste à :

- **Préparer un jeu de données de test**
- **Appliquer le modèle** construit sur les **variables explicatives**
- **Comparer les prédictions** avec les valeurs réelles du **label**
- **Calculer la pertinence** pour évaluer la **performance** du modèle

III. Apprentissage Supervisé vs Non Supervisé

a. Apprentissage Supervisé (Classification)

- Les **données d'apprentissage** (**observations**) sont accompagnées des **labels** indiquant leurs **classes**.
- Les **nouvelles données** sont **classifiées** en se basant sur le **jeu de données d'apprentissage** (**training set**).

b. Apprentissage Non Supervisé (Regroupement)

- Les **labels** des éléments du **jeu de données** ne sont pas connus.
- Le but est de **détecter l'existence de classes ou groupes** dans les données.

IV. Classification avec Arbres de Décision

a. Arbre de Décision

Les **nœuds internes** correspondent à des **tests**.

Un **arc** correspond au **résultat d'un test**.

Les **nœuds feuilles** représentent des **classes**.

b. Génération de l'arbre

i. Construction de l'arbre

Au début, tous les **tuples** se trouvent sur la **racine**.

Les **tuples** sont **partitionnés récursivement** en se basant sur un **attribut sélectionné** à chaque étape.

ii. Simplification de l'arbre

Identifier et **supprimer les branches** qui correspondent à des **exceptions**.

c. Utilisation de l'arbre

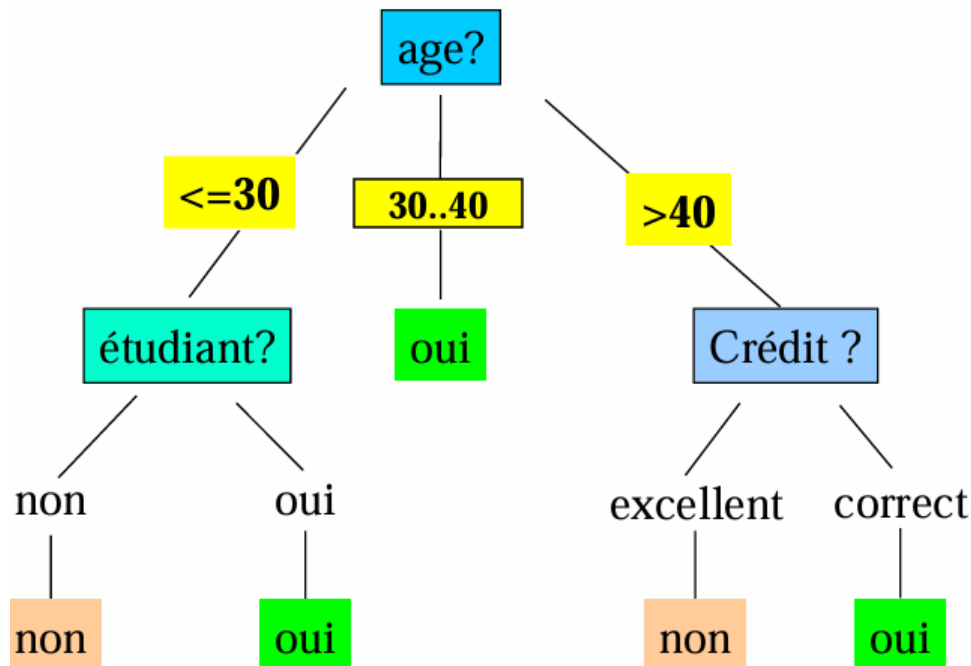
Tester les **attributs** d'un **tuple** par rapport à l'**arbre** pour suivre la **branche correspondante** et déterminer la **classe**.

Ex : Classification avec Arbre de Décision : Exemple "achète_ordinateur"

1. Jeu de données (Training Set)

âge	salaire	étudiant	crédit	achète_ordinateur
<=30	élevé	non	correct	non
<=30	élevé	non	excellent	non
30...40	élevé	non	correct	oui
>40	moyen	non	correct	oui
>40	faible	oui	correct	oui
>40	faible	oui	excellent	non
31...40	faible	oui	excellent	oui
<=30	moyen	non	correct	non
<=30	faible	oui	correct	oui
>40	moyen	oui	correct	oui
<=30	moyen	oui	excellent	oui
31...40	moyen	non	excellent	oui
31...40	élevé	oui	correct	oui
>40	moyen	non	excellent	non

Arbre de décision complet pour achète_ordinateur



Interprétation Arbre de décision pour achète_ordinateur

1. Racine : âge

a. Si âge <= 30

- Attribut suivant : **étudiant ?**
 - Oui → Classe : **oui**
 - Non → Classe : **non**

b. Si âge 31...40

- Attribut suivant : **étudiant ?**
 - Oui → Classe : **oui**
 - Non → Classe : **oui**

c. Si âge > 40

- Attribut suivant : **crédit ?**
 - Crédit = **excellent** → Classe : **non**
 - Crédit = **correct** → Classe : **oui**

Cet arbre permet de **prédire la valeur de "achète_ordinateur"** en fonction de **âge**, **étudiant**, et **crédit**.

V. Création de l'arbre de décision

- Un arbre de décision est construit de manière **top-down réursive**.

Définition : "Top-down récursif" signifie que l'arbre est construit **du sommet vers les feuilles** et que le même processus de partitionnement est **appliqué à chaque sous-ensemble de données** de façon répétée jusqu'à obtenir des feuilles homogènes.

- Au début, tous les **tuples** sont placés sur la **racine**.
- Les **attributs** sont généralement **qualitatifs** (si nécessaire, on effectue une **discrétisation**).
- Les **tuples** sont ensuite **partitionnés** en fonction de l'**attribut sélectionné**.
 - Partitionnement : Séparation des données en sous-ensembles.
- L'**attribut de test** est choisi en utilisant des **heuristiques**, par exemple le **gain informationnel**.
 - Heuristiques : méthode utilisée pour guider le choix de l'attribut à tester à chaque nœud.
- Conditions d'arrêt** du partitionnement :
 - Tous les **tuples** d'un **nœud** appartiennent à la **même classe**.

Ainsi, l'arbre se construit progressivement, en testant successivement les attributs qui permettent de mieux séparer les données jusqu'à obtenir des feuilles homogènes.

a. Choix de l'attribut de partitionnement

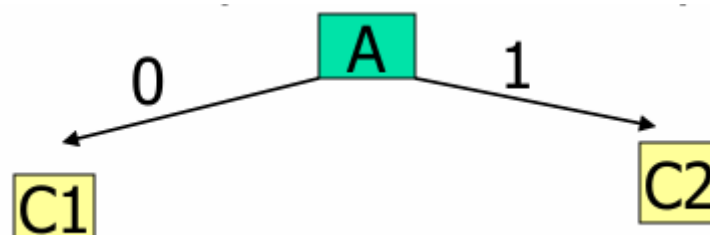
1. Jeu de données (Training Set)

A	B	Classe
0	1	C1
0	0	C1
1	1	C2
1	0	C2

2. Partitionnement selon l'attribut A

- $A = 0 \rightarrow$ tuples : $(0,1,C1), (0,0,C1) \rightarrow$ toutes **C1** $\rightarrow$ **nœud terminal** = C1
- $A = 1 \rightarrow$ tuples : $(1,1,C2), (1,0,C2) \rightarrow$ toutes **C2** $\rightarrow$ **nœud terminal** = C2

Arbre obtenu :

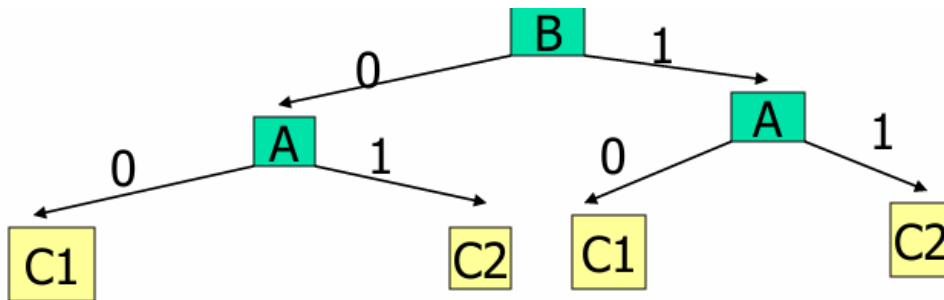


Si c'est A qui est choisi en 1er

3. Partitionnement selon l'attribut B

- $B = 0 \rightarrow$ tuples : $(0,0,C1), (1,0,C2) \rightarrow$ mélange **C1 et C2** $\rightarrow$ partitionner avec A
- $B = 1 \rightarrow$ tuples : $(0,1,C1), (1,1,C2) \rightarrow$ mélange **C1 et C2** $\rightarrow$ partitionner avec A

Arbre obtenu après deuxième étape :



Si c'est B qui est choisi en 1er

4. Interprétation

- Choisir **A** en premier sépare directement les classes $\rightarrow$ arbre **court et clair**.
- Choisir **B** en premier nécessite une deuxième étape $\rightarrow$ arbre **plus long et complexe**.
- Les **heuristiques** comme le **gain informationnel** permettent de choisir l'attribut qui sépare **le mieux les classes dès le premier nœud**.

b. Choix de l'attribut de partitionnement (2)

- Un **arbre de décision** représente la **suite de questions** à poser pour **classifier un nouvel exemple**.
- Le but est d'obtenir une **classification correcte** en posant le **moins de questions possible**.
- Dans l'exemple précédent, l'attribut **A** apporte plus d'**information** pour classifier les exemples que l'attribut **B**.
- Il est donc nécessaire de **quantifier l'information** apportée par chaque **attribut** afin de choisir le meilleur pour chaque nœud.

Objectif : Avoir un arbre le plus **compact** possible.

Pas de souci, on va tout détailler **pas à pas**, comme si on partait de zéro.

V. Notion sur la théorie de l'information

a. L'information d'un événement

- **Idée principale** : plus un événement est **probable**, moins il nous apprend quelque chose de nouveau.
- **Exemple concret** : si tu es dans le désert et qu'on te dit qu'il fera beau demain... c'est évident, donc ça **n'apporte presque rien comme information**.

- Pour formaliser ça, on définit une **fonction $h(X)$** qui mesure l'information d'un événement (X) en fonction de sa **probabilité** :

$$h(X) = f\left(\frac{1}{\text{Proba}(X)}\right)$$

- Si un événement est **certain** (il arrive toujours), la fonction doit donner **0**, car il n'y a **aucune surprise** : ($f(1) = 0$).

b. Information de plusieurs événements indépendants

- Si tu as **deux événements qui ne dépendent pas l'un de l'autre** (indépendants), la quantité d'information totale est la **somme de leurs informations individuelles**.
- Exemple : lancer deux pièces de monnaie.
 - Obtenir "face" sur la première = info1
 - Obtenir "pile" sur la deuxième = info2
 - Les deux ensembles = info1 + info2
- Mathématiquement :

$$h(X, Y) = f\left(\frac{1}{\text{Proba}(X, Y)}\right) = f\left(\frac{1}{\text{Proba}(X) * \text{Proba}(Y)}\right) = h(X) + h(Y)$$

- Pour que ça marche bien, on utilise **le logarithme en base 2** :

$$h(X) = -\log_2(\text{Proba}(X))$$

Pourquoi ?

1. Si la probabilité est faible → plus d'information → la fonction **augmente quand la probabilité diminue**
2. Deux événements indépendants → somme des informations → règle respectée

c. Information pour décider dans une classification binaire

- Imagine que tu as **deux classes** :
 - **P** (positif)
 - **N** (négatif)
- Tu as un ensemble (S) avec :
 - (p) éléments de **P**
 - (n) éléments de **N**

- La **probabilité qu'un élément choisi au hasard soit dans P** est :

$$p/(p+n)$$

- La **quantité d'information nécessaire** pour savoir si un élément est **P ou N** se calcule avec l'**entropie** :

$$I(p, n) = -p \log_2 \frac{p}{p+n} - n \log_2 \frac{n}{p+n}$$

- Pourquoi cette formule ?
 1. Chaque partie ($-p \log p$ et $-n \log n$) mesure **combien de surprise** il y a si tu tires un élément de cette classe.
 2. On fait la somme → on obtient l'**incertitude totale** de l'ensemble.
- Plus l'entropie est **grande**, plus on **ne sait pas à quelle classe appartient un élément** → il faudra plus d'information pour le classifier.
- Plus l'entropie est **petite**, plus la classification est **facile**, l'attribut choisi sépare bien les classes.

VI. Extraction des règles de classification

a. Principe général

- Les **règles de classification** sont de la forme **SI – ALORS (IF – THEN)**.
- Chaque **chemin** partant de la **racine** d'un arbre de décision jusqu'à une **feuille** correspond à **une règle**.
- Chaque **paire attribut–valeur** rencontrée le long de ce chemin forme une **conjonction** (*ensemble de conditions reliées par des « ET »*).
- Les **feuilles** indiquent la **classe finale prédite** (la valeur du **label**).

VII. Problème de l'overfitting

a. Définition et explication

Lors de la **construction d'un modèle de classification**, il est possible que l'arbre obtenu **classe parfaitement** les exemples du **training set** (ensemble d'apprentissage).

Cela signifie qu'il **n'y a aucune erreur** sur ces données, car le modèle a appris **tous les détails**, même ceux qui sont **spécifiques ou accidentels**.

Cependant, un arbre qui **colle trop** aux données d'apprentissage risque de **mal généraliser** lorsqu'il est confronté à de **nouvelles données**.

Ce phénomène est appelé **overfitting** (ou **surapprentissage**).

b. Conséquence

Un modèle en **overfitting** :

- **Apprend trop bien** les données d'apprentissage, y compris les **bruits ou anomalies** ;
- Devient **trop complexe**, avec **trop de branches** et de conditions ;
- Perd sa **capacité à prédire correctement** sur d'autres ensembles de données (dits de **test**).

c. Solution

Pour **éviter l'overfitting**, on accepte volontairement **quelques erreurs** sur le **training set**, afin de construire un arbre **plus simple et plus général**.

L'objectif est d'obtenir un **modèle équilibré**, capable de :

- **bien apprendre** les tendances générales,
- mais aussi **s'adapter** à de nouvelles observations.

d. Exemple

SI âge = "≤ 30" **ET** étudiant = "non" **ET** revenu = "moyen" **ET** habite = "centre-ville" **ET** aime_technologie = "non"

ALORS achète_ordinateur = "non"

→ Cet arbre est **trop précis** (chaque détail du training set est pris en compte).

Il fonctionnera très bien sur le jeu d'apprentissage, mais **échouera sur de nouveaux cas**.

Une **version plus simple et plus robuste** serait :

SI âge = "≤ 30" **ET** étudiant = "non"

ALORS achète_ordinateur = "non"

VIII. Généraliser l'arbre induit

a. Objectif

Afin d'éviter le **problème de l'overfitting**, il est nécessaire de **généraliser l'arbre de décision**. Cela signifie qu'on cherche à **simplifier** l'arbre tout en **conservant une bonne précision**. Deux approches principales permettent cette généralisation : le **prepruning** et le **postpruning**.

b. Approche 1 : Prepruning (Élagage précoce)

Cette méthode consiste à **arrêter la construction de l'arbre** avant qu'il ne devienne trop complexe.

- On **ne découpe pas un nœud** si le **gain** (ou la **mesure de pertinence**) devient **inférieur à un seuil défini**.
- Cela permet de **limiter le surapprentissage** en évitant la création de branches inutiles.

Avantage :

L'arbre est **directement plus simple** et **rapide à construire**.

Inconvénient :

Il est **difficile de choisir le seuil optimal** — un seuil trop élevé peut **stopper trop tôt** la construction et **perdre en précision**.

c. Approche 2 : Postpruning (Élagage a posteriori)

Cette méthode consiste à **construire d'abord l'arbre complet**, puis à **supprimer les branches inutiles**.

- On **teste plusieurs versions réduites** de l'arbre.
- On utilise un **ensemble de données différent du training set** (souvent un **jeu de validation**) pour **évaluer** chaque arbre réduit.
- L'arbre retenu est celui qui **généralise le mieux** (meilleur compromis entre simplicité et précision).

Avantage :

L'arbre final est **plus robuste** face aux nouvelles données.

Inconvénient :

Le processus est **plus long** car il nécessite des **tests supplémentaires**.

d. Exemple

Un arbre complet pourrait contenir une branche comme :

SI âge = " ≤ 30 " **ET** étudiant = "oui" **ET** revenu = "moyen" **ALORS** achète_ordinateur = "oui"

Lors du **postpruning**, si l'ajout du critère "revenu = moyen" n'améliore pas significativement la précision, on **supprime** cette condition, pour obtenir une règle plus générale :

SI âge = " ≤ 30 " **ET** étudiant = "oui" **ALORS** achète_ordinateur = "oui".

IX. Classification Bayésienne

a. Principe général

La **classification bayésienne** repose sur la **théorie des probabilités**.

Elle cherche à **prédire la classe d'un objet** non pas de manière absolue, mais **en termes de probabilité**.

Autrement dit, elle **pondère plusieurs hypothèses** selon leur **probabilité d'occurrence**.

b. Fonctionnement

1. On considère un **objet O** (ou un individu) dont on connaît certaines **caractéristiques**.
2. Pour chaque **classe possible**, on **calcule la probabilité d'appartenance** de O à cette classe.
3. On **choisit la classe** qui **maximise cette probabilité** — c'est-à-dire la classe **la plus probable** pour cet objet.

Cette approche repose sur le **théorème de Bayes**, qui relie les probabilités conditionnelles :

$$P(\text{Classe} \mid \text{Données}) = (P(\text{Données} \mid \text{Classe}) \times P(\text{Classe})) / P(\text{Données})$$

- **$P(\text{Classe} \mid \text{Données})$** : probabilité que l'objet appartienne à une classe donnée (ce que l'on cherche à estimer)
- **$P(\text{Données} \mid \text{Classe})$** : probabilité d'observer ces données si l'objet appartient à cette classe
- **$P(\text{Classe})$** : probabilité initiale (ou a priori) de cette classe
- **$P(\text{Données})$** : probabilité d'observer ces données toutes classes confondues

c. Exemple

Supposons qu'on veuille prédire si un étudiant réussira à un examen.

Les classes sont : **{Réussite, Échec}**.

Les données observées sont : le **nombre d'heures de révision** et la **présence aux cours**.

- On calcule **$P(\text{Réussite} \mid \text{Données})$** et **$P(\text{Échec} \mid \text{Données})$** .
- Si **$P(\text{Réussite} \mid \text{Données}) > P(\text{Échec} \mid \text{Données})$** , on classera l'étudiant dans la catégorie "**Réussite**".

d. Remarques importantes

- Même si les méthodes bayésiennes peuvent être **complexes à calculer** (voire **intractables** dans certains cas), elles servent souvent **d'étalon de référence** pour évaluer la **performance** et la **correction** d'autres méthodes de classification.
- Une variante très utilisée est le **classifieur naïf bayésien (Naive Bayes)**, qui suppose que les **attributs sont indépendants** entre eux — simplifiant ainsi grandement le calcul des probabilités.

- Soit le *training set* D , la probabilité a posteriori de l'hypothèse h , $P(h|D)$ suit le théorème de Bayes

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

- MAP (maximum posteriori) hypothesis

$$h_{MAP} \equiv \underset{h \in H}{\text{Max}} P(h | D) = \underset{h \in H}{\text{Max}} P(D | h)P(h).$$

Théorème de Bayes

- **Théorème de Bayes :**

$$P(C|X) = P(X|C) \cdot P(C) / P(X)$$

- $P(X)$ est la même pour toutes les classes
- $P(C)$ = fréquence relative des éléments de C
- C telle que $P(C|X)$ est maximum =
 C telle que $P(X|C) \cdot P(C)$ est maximum
- Problème: calculer $P(X|C)$ est infaisable !